



A reinforcement learning level-based particle swarm optimization algorithm for large-scale optimization

Feng Wang^{*}, Xujie Wang, Shilei Sun^{*}

School of Computer Science, Wuhan University, Wuhan 430072, China

ARTICLE INFO

Article history:

Received 13 December 2021

Received in revised form 19 March 2022

Accepted 24 April 2022

Available online 29 April 2022

Keywords:

Reinforcement learning

Particle swarm optimization

Large-scale optimization

ABSTRACT

Large-scale optimization problems (LSOPs) have drawn researchers' increasing attention since their resemblance to real-world problems. However, due to the complex search space and massive local optima, it is challenging to simultaneously guarantee the diversity and convergence of the algorithms. As a widely used evolutionary algorithm with fast convergence, particle swarm optimization (PSO) shows competitive performances on some LSOPs. Nevertheless, it can easily get trapped into local optima. Overcoming the complexity of LSOPs and improving search efficiency have become vital issues. The reinforcement learning method has proven to be an effective technique in self-adaptive adjustment, which can help search for better results in large-scale solution space more effectively. In this paper, we propose a large-scale optimization algorithm called reinforcement learning level-based particle swarm optimization algorithm (RLLPSO). In RLLPSO, a level-based population structure is constructed to improve population diversity. A reinforcement learning strategy for level number control is employed to help improve the search efficiency of RLLPSO. To further enhance the convergence ability of RLLPSO, a level competition mechanism is introduced. The experimental results from two large-scale benchmark test suites demonstrate that, compared with five state-of-the-art large-scale optimization algorithms, RLLPSO shows superiority in most cases.

© 2022 Elsevier Inc. All rights reserved.

1. Introduction

The large-scale optimization problem is named after its large dimensionality, which can be defined as follows:

$$\min f(X), X = [x^1, x^2, \dots, x^D] \quad (1)$$

where D stands for the numbers of dimensions ($D \geq 500$ in LSOPs), and $f(X)$ is the fitness value of the corresponding solution X . As the scale of the problems increases, the performances of the algorithms deteriorate rapidly, which can be due to two main reasons. First, higher dimensionality makes the search space more complex and enormous, which reduces the optimization algorithm's search efficiency and results in slow convergence. Second, there are massive local optima in LSOPs, which can easily make the population suffer premature problems. Therefore, the major challenges that the algorithms for LSOPs need to conquer are how to efficiently detect the search space and maintain population diversity to escape from the local optima. Attracted by the real-world application potential of LSOPs, researchers have adopted plenty of methods to solve them [18,28,30,43,44]. PSO, as an effective evolutionary algorithm which has been applied to numerous optimiza-

^{*} Corresponding authors.

E-mail addresses: fengwang@whu.edu.cn (F. Wang), sunsl@whu.edu.cn (S. Sun).

tion problems, has also shown great potential in LSOPs [5,32,33,45]. Recently, researchers have designed many algorithms for large-scale optimization, such as dimensionality reduction based algorithms and integrality strategy based algorithms.

The most common strategy for large-scale optimization algorithms is dimensionality reduction, which divides a high-dimensional problem into some low-dimensional problems [8]. Some methods decompose the problems before the search process, and the sub-populations are fixed during the optimization [6,20,23,24]. Since it is difficult to design a strategy that can appropriately decompose all problems, some other methods introducing mechanisms that can dynamically adjust the size of sub-populations during the search process have also been proposed [16,17,25,42]. Nevertheless, the efficiency of these algorithms severely relies on the effectiveness of the decomposition strategies. Furthermore, they cannot work well on non-separable problems and some partially separable problems. During the decomposition process, the effectiveness of such algorithms has been affected by problems such as information loss and extra computation resource consumption.

To avoid information loss during the decomposition process, other algorithms that consider all dimensions of the LSOPs integrally are investigated, which is denoted as integrality strategy based algorithms. To fully detect the complex search space efficiently and escape from the numerous local optima, it is required to keep a good population diversity for such algorithms. Some algorithms have adopted specially designed updating strategies to improve the exploration performances [1,2,9,21,28,41], while some other algorithms have introduced distributed strategies for maintaining population diversity [36,37]. Recently, some algorithms developed multiple strategies for different situations in LSOPs and invoke them accordingly [3,7,34]. As the convergence speed decreases in the vast and complex search space of LSOPs, maintaining the balance between exploration and exploitation mainly affects the performance of the above algorithms.

Reinforcement learning (RL) is a useful machine learning technique, which solves problems by interacting with the environment through carefully designed learning strategies [31,35]. Compared with other adjustment strategies, RL accumulates the influence of historical information for better judgments, which makes the decisions more robust. Additionally, RL can be adequately applied to various problems while requiring no expert experience. It can learn a specific policy or rule for different search environments during iterations. Benefiting from RL, the evolutionary algorithms (EAs) [12,13,29] can predict and choose the most promising actions in the current state by maximizing the numerical reward signal. Therefore, some RL based evolutionary algorithms have been proposed [22,26]. The results indicate that RL can enhance the efficiency and stability of EAs.

To improve the search efficiency and convergence performance for LSOPs, this paper proposes a reinforcement learning level-based particle swarm optimization algorithm (RLLPSO). Specifically, the major contributions of this paper are summarized as follows.

- 1) A level-based population structure is adopted to solve large-scale optimization problems integrally in RLLPSO. Based on such structure, particles in lower levels select particles in higher levels as their learning exemplars instead of the historical best positions of the population, which enhances the diversity of RLLPSO.
- 2) A reinforcement learning strategy for level number control is introduced to self-adaptively adjust the level structure in the population instead of manual control. Due to the flexibility and adaptability ensured by the RL mechanism, RLLPSO can improve search efficiency by taking the most potential actions based on the environment reward.
- 3) To optimize the convergence performance of RLLPSO, a level competition mechanism is proposed. It allows RLLPSO to choose particles in more elite levels as learning exemplars with a higher probability. Thus, it can help the population converge more rapidly and reduce computation costs.

The remainder of this paper is organized as follows. Section 2 briefly introduces the traditional PSO, its variants, and some work based on reinforcement learning. Section 3 elucidates our proposed algorithm in detail. The experiment results and analysis are given in Section 4. Finally, the conclusion and the discussion of future work are presented in Section 5.

2. Related work

2.1. Traditional PSO

Inspired by some swarm behaviors such as predatory behaviors in flocks of birds, Kennedy and Eberhart proposed PSO [5]. As one of the most popular EAs, each member of the population is regarded as a particle that stands for a solution of the optimization problems. Each particle contains a velocity vector $V_i = [v_i^1, v_i^2, \dots, v_i^D]$ and a position vector $X_i = [x_i^1, x_i^2, \dots, x_i^D]$, and D stands for the number of the variables. In addition, PSO also stores the personal historical best position of particles and the historical best position of the entire population, called the personal best $P_i = [p_i^1, p_i^2, \dots, p_i^D]$ and the global best $P_g = [p_g^1, p_g^2, \dots, p_g^D]$, respectively. During the iterations, PSO will update vectors of the particles by the following rules:

$$v_i^d(t+1) = wv_i^d(t) + c_1r_1(p_i^d(t) - x_i^d(t)) + c_2r_2(p_g^d(t) - x_i^d(t)) \quad (2)$$

$$x_i^d(t+1) = x_i^d(t) + v_i^d(t+1) \quad (3)$$

where v_i^d and x_i^d represent the velocity and the current position of the i^{th} particle on the d^{th} dimension. r_1 and r_2 are two random numbers which uniformly distributed within the range of $[0,1]$. w is an inertia factor that balances global and local search preference. c_1 and c_2 are the accelerate coefficients which determine the particle movement preference to either P_i or P_g . p_i^d is the position of P_i on the d^{th} dimension, and p_g^d represents the position of the global best particle P_g on the d^{th} dimension.

2.2. PSO variants for large-scale optimization

Traditional PSO has shown good performance on some small-scale problems with low dimensionality. With the increasing dimensionality, its performance becomes limited. To overcome the enormous search space limitation of the LSOPs, some dimensionality reduction strategies have been applied in PSO variants. As it will cause search space information loss and extra computation cost, other approaches that optimize all dimensions integrally are proposed later.

2.2.1. Dimensionality reduction based PSO variants

The complex search space in high-dimensional problems causes exploration difficulty during the search process. A common method called dimensionality reduction was introduced to address the problem by dividing it into more minor problems with fewer dimensions and then conquering them separately. With lower dimensionality, the search space becomes simpler, which algorithms can search more efficiently.

Some variants determine the division situation before the optimization. A cooperative coevolution algorithm (CCEA) created by Potter et al. adopts a CC framework that considers every variable as a sub-component of the problem and conquers them respectively [24]. Based on CC framework, other PSO variants such as CPSO [6], CCPSO [16], and CCPSO2 [17] have been proposed. Like CCEA, CPSO- S_K divides the population into K sub-populations before the search process. And CPSO- H_K combines the traditional PSO and CPSO- S_K and invokes them alternately. However, a fixed sub-population strategy cannot deal with all LSOPs efficiently. Therefore, some variants that allow the population to adjust the sub-population structure dynamically are proposed. DSPLSO changes its sub-components size after every iteration that fails to improve the performance of the population [42]. In CCPSO [16] and CCPSO2 [17], an adaptive scheme dynamically controls the subcomponent sizes of the problem. They also introduce a randomly divide strategy, which allows the correlative dimensions to be decomposed into the same sub-population from time to time during the iterations. Hence, the performance on some partially separable functions can be ensured.

Although these PSO variants show competitive performance on some LSOPs, there are still some remaining problems. Firstly, the algorithm's effectiveness is highly dependent on the decomposition strategies, but no strategy can handle all the high-dimensional problems perfectly. Secondly, the dimensionality reduction strategy can only be applied to separable problems and some partially separable problems. The performance of such algorithms is poor on other partially separable problems which cannot be appropriately decomposed and non-separable problems. Thirdly, the information loss during the dimensionality reduction process can cause problems such as global optima disappearance. Last but not least, because of the sub-swarm evolution, some decomposition strategies may require extra computation resources, and the extra resources consumption will weaken the search efficiency of the algorithm in LSOPs.

2.2.2. Integrality strategy based PSO variants

Some PSO variants with different integrality strategies that consider all dimensions together are proposed to avoid information loss during the decomposition process and enhance performances on some partially separable and non-separable problems. These PSO variants can search the immense space of high-dimensional problems efficiently with fast convergence speed. Nevertheless, the premature problem severely influences their performances.

Some algorithms employ different particle updating strategies to improve population diversity [28]. For example, instead of historical best positions, in SL-PSO [2], particles learn from the other particles with higher fitness values. The higher the fitness value a particle has, the more likely it will be chosen. CSO [1] randomly chooses two particles from the swarm, and the worse particle will move towards the better particle during the iterations. Additionally, some algorithms such as DLLSO [41] and DSPLSO [42] introduce different strategies that select two different better particles from the swarm as the learning exemplars. Furthermore, some algorithms introduce multi-swarm strategies which decompose the swarm into sub-swarms, such as DGLDPSO and AGLDPSO [36,37,46]. Each sub-swarms evolves and detects the search space in parallel and exchanges valuable information with each other during the iterations to ensure convergence efficiency. It has proven to be effective to combine multiple strategies and introduce local search mechanisms in large-scale optimization [3,7,9,34]. And after that, algorithms can solve the LSOPs efficiently with multiple strategies designed to handle various situations.

Due to large-scale optimization's massive and complex search space, it is challenging to explore and efficiently converge simultaneously for LSOPs. The major challenges of LSOPs for integrality strategy based algorithms are still slow convergence and falling into local optima. The strategies that improve the diversity will influence convergence efficiency in the later optimization stage, and variants adopted multiple strategies for different situations cannot always choose the proper strategy.

2.3. Reinforcement learning for evolutionary algorithms

Unlike supervised and unsupervised learning, reinforcement learning (RL) requires no data in advance for optimization problems. It can obtain learning information and update model parameters from the environment via states and actions which are reasonably designed. As it can achieve self-adaptive adjustment through learning strategies, researchers have made some efforts that combine RL with EAs. Researchers find that combining EAs with RL and using RL to handle the tricky parameters setting problem or the proper strategy selection is a sterling approach [4,10,11,22,26,38].

Some algorithms employ RL to help EA adjust its parameters [11]. In RLMPSO [26], RL-MOEA/D [22], and QLPSO [19], RL is introduced as a strategy control mechanism that allows the algorithms to determine updating strategies automatically. Furthermore, in RL-MOEA/D, the neighborhood size is also monitored and adjusted by an RL controller. Besides, RL can also be used to adjust the topology structure of PSO [40]. In complex real-world problems, for instance, robot control and coal mining [14], RL also cooperates with EAs well [10] [38]. Although the performance of EAs combined with RL has proven to be efficient in traditional optimization problems and some real-world problems, its application in LSOPs has not been fully noticed.

The above research indicate that RL can help EAs in determining correct update strategies or adjusting parameters. RL mechanism can eliminate the uncertainty of manual control and improve the search performance of EAs in complex situations.

3. Reinforcement learning level-based particle swarm optimization algorithm (RLLPSO)

It is well known that the search space expands and complicates exponentially as the dimensionality increases in LSOPs. Moreover, the tremendous search space requires good search efficiency while the complex situation challenges the exploration ability. To improve the search efficiency of PSO for LSOPs, we proposed a reinforcement learning level-based particle swarm optimization algorithm (RLLPSO).

To maintain good diversity, RLLPSO adopts a level-based population structure [41]. In RLLPSO, the swarm containing NP particles is equally divided into l levels after sorting by the fitness value. The particle number in every level is $LS = NP/l$. And in the last level, there are $NP/l + NP\%l$ particles. Particles in the worse levels will choose two random particles from two different random better levels as their exemplars during the iterations. Then, the particles will update themselves based on Eq. (4) and Eq. (5):

$$v_{ij}^d(t+1) = r_1 v_{ij}^d(t) + r_2 (x_{le_1,k_1}^d(t) - x_{ij}^d(t)) + \phi r_3 (x_{le_2,k_2}^d(t) - x_{ij}^d(t)) \quad (4)$$

$$x_{ij}^d(t+1) = x_{ij}^d(t) + v_{ij}^d(t+1) \quad (5)$$

where $x_{ij}^d(t)$ is the value of the j^{th} particle from the i^{th} level l_i on the d^{th} dimension in the t^{th} iteration, and $v_{ij}^d(t)$ is its speed. $x_{le_1,k_1}^d(t)$ and $x_{le_2,k_2}^d(t)$ are the values of two exemplars chosen from level le_1 and le_2 , where $le_1 < le_2 < i$. k_1 and k_2 are two random integers that indicates the indexes of particles in level le_1 and le_2 , respectively. r_1, r_2 and r_3 represents three numbers randomly obtained from range $[0, 1]$. ϕ , which controls the influence of the less elite exemplars, is a parameter within $[0, 1]$. During the iterations, the particles of the first level will not be updated, and the particles of the second level will randomly choose both of its exemplars from the first level.

3.1. Reinforcement learning strategy for level number control

In RLLPSO, particles in the swarm are divided into l levels. The particles from lower levels have more potential learning exemplars, which indicate higher diversity, while the particles from the higher levels focus more on exploitation. The search preference of the population can be affected by the constantly adjusting level structure during the evolutionary process. To improve the search efficiency of RLLPSO, a reinforcement learning strategy for level number control is introduced, which can help choose a proper level number l based on the historical information and environment reward.

The level number will be updated after every iteration. The candidate choices are stored in a candidate pool $S = \{l_1, l_2, \dots, l_s\}$. As shown in Fig. 1, by considering the level number candidates as both states and actions, a Q-table is constructed and maintained by the Q-learning algorithm [39].

The initial Q-values of each state and its actions are set at 0. During the optimization, when the current state is l_c , the following action $action_{next}$ is determined by the Q-table with the following rules.

$$action_{next} = \begin{cases} \text{Argmax}[Q(l_c, action_c)], & \text{rand}() < \epsilon \\ action_{random}, & \text{otherwise} \end{cases} \quad (6)$$

where $action_c$ is any action the swarm can take in current state l_c , and $\text{Argmax}[Q(l_c, action_c)]$ represents the action with the highest Q-value. ϵ is a control parameter that prevents the action selection from premature. $action_{random}$ is a randomly selected action.

After an action is determined, RLLPSO will execute an iteration. Because of the search randomness of the algorithm, it is normal to fail to update P_g between iterations. Therefore, there is no need to heavily punish an action with a negative reward

state /action	l_1	l_2	l_3	l_4	l_5	l_6
l_1	0.165	0.022	0.005	0.365	0.512	0.227
l_2	0.042	0.491	0.388	0.25	0.176	0.632
l_3	0.477	0.393	0.332	0.145	0.724	0.532
l_4	0.289	0.521	0.236	0.33	0.365	0.229
l_5	0.463	0.223	0.298	0.015	0.096	0.169
l_6	0.576	0.231	0.335	0.284	0.123	0.256

$r_{1,5} = 0.25$

state /action	l_1	l_2	l_3	l_4	l_5	l_6
l_1	0.165	0.022	0.005	0.365	0.555	0.227
l_2	0.042	0.491	0.388	0.25	0.176	0.632
l_3	0.477	0.393	0.332	0.145	0.724	0.532
l_4	0.289	0.521	0.236	0.33	0.365	0.229
l_5	0.463	0.223	0.298	0.015	0.096	0.169
l_6	0.576	0.231	0.335	0.284	0.123	0.256

Fig. 1. The Q-table update process.

when such a situation occurs. After updating the particles, instead of a fixed number, a reward r_{ij} denoted as Eq. (7) will be obtained from the population, which represents the influence of selecting l_j as the action when the current population state is l_i .

$$r_{ij} = \frac{|F_g(t) - F_g(t+1)|}{|F_g(t)|} \quad (7)$$

where $F_g(t)$ and $F_g(t+1)$ represent the fitness value of P_g in the t^{th} and $(t+1)^{th}$ iteration respectively.

After the reward is received, to keep the effectiveness of the action selection strategy, the reinforcement learning strategy will update its Q-value by the following rule:

$$Q(i,j) = Q(i,j) + \alpha \cdot (r_{ij} + \gamma \cdot \text{Max}(Q(j)) - Q(i,j)) \quad (8)$$

where $Q(i,j)$ is the approximate expectation of the reward that selects l_j as the next state when the current population state is l_i . And $\text{Max}(Q(j))$ denotes the max Q-value of the next swarm state l_j in the Q-table. Fig. 1 presents an example of how the reinforcement learning strategy for level number control works in RLLPSO. As shown in Fig. 1, when $\alpha = 0.4$ and $\gamma = 0.8$, the Q-table will update after a reward $r_{1,5}$ is received. $\text{Max}(Q(j))$ means $Q(5,1)$ here. Algorithm 1 shows the details of the reinforcement learning strategy for level number control.

Algorithm 1 Reinforcement learning strategy for level number control

```

1: Initialize Q-table;
2: while stop condition is not reached do
3:   if  $\text{rand}() < \epsilon$  then
4:      $\text{action}_{\text{next}} = \text{Argmax}[Q(l_c, \text{action}_c)]$ ;
5:   else
6:      $\text{action}_{\text{next}} = \text{action}_{\text{random}}$ ;
7:   end if
8:   Execute  $\text{action}_{\text{next}}$ , receive a reward  $r_{ij} = \frac{|F_g(t) - F_g(t+1)|}{|F_g(t)|}$ ;
9:   Update Q-table:  $Q(i,j) = Q(i,j) + \alpha \cdot (r_{ij} + \gamma \cdot \text{Max}(Q(j)) - Q(i,j))$ ;
10: end while

```

3.2. Level competition mechanism

Through a level structure swarm, RLLPSO can keep good population diversity. Nevertheless, the random exemplar selection restrains its convergence efficiency. To increase the convergence ability, a level competition mechanism is designed to give elite levels a higher probability of being chosen as the learning exemplar level. The parameter prob_l is the probability of the level competition mechanism being triggered during the iterations:

$$\text{prob}_l = \left(\frac{FE_{\text{count}}}{\text{Max}_{FEs}} \right)^2 \quad (9)$$

where FE_{count} is the computation resources that the algorithm has used, and Max_{FEs} is the computation budget. Fig. 2 shows that how $prob_l$ changes with FE_{count} .

The level competition mechanism is described in Algorithm 2. For every updating particle from level le_c , when the level competition mechanism is triggered, RLLPSO will randomly choose two candidate levels to make a competition instead of randomly choosing a higher level as the learning target level. The winner level will be selected as the target level le_i , and one exemplar will be chosen from le_i randomly. As a result, the particles with higher fitness values and more valuable solution information are more likely to be selected as the learning exemplars. Furthermore, as the level competition mechanism is rarely triggered at the beginning of the optimization, the diversity of the algorithm can be maintained, and the level competition mechanism improves the convergence efficiency of the algorithm in the later stage.

Algorithm 1 Level competition mechanism

Input: Input: the level of the updating particle, le_c ;

```

1:  $prob_l = \left(\frac{FE_{count}}{Max_{FEs}}\right)^2$ ;
2: for  $i = 1 \rightarrow 2$  do
3:   if  $rand() < prob_l$  then
4:     Randomly select two level higher than  $le_c$ ,  $le_{i,c1}$  and  $le_{i,c2}$ ;
5:     Compare  $le_{i,c1}$  and  $le_{i,c2}$ , select the higher level as the exemplar level  $le_i$ ;
6:   else
7:     Randomly choose one level higher than  $le_c$  as the exemplar level  $le_i$ ;
8:   end if
9: end for
10: if  $le_2$  is higher than  $le_1$  then
11:   Swap  $le_1$  and  $le_2$ ;
12: end if
Output: Output: exemplar levels  $le_1, le_2$ 

```

3.3. Framework of RLLPSO

The framework of RLLPSO is presented as in Algorithm 3.

Algorithm 1 Framework of RLLPSO

```

1: Initialize positions and velocities of all particles within the search space;
2: Initialize  $FE_{count} = 0, prob_l = 0, s_0 = l_1$ ;
3: Evaluate the fitness value of every particle and update  $F_g$ ;
4: while stop condition is not reached do
5:   Use the Algorithm 1 to determine next action  $a_{t+1}$ ;
6:   Sort the swarm by fitness value and divide it into  $a_{t+1}$  levels;
7:   for  $i = 3 \rightarrow a_{t+1}$  do
8:     for  $j = 1 \rightarrow LS$  do
9:       Use Algorithm 2 to select two levels  $le_1, le_2$  as the learning target levels;
10:      Randomly select two particles  $X_{le_1,k_1}, X_{le_2,k_2}$  from  $le_1, le_2$ ;
11:      Update the  $X_{ij}(t)$  and  $V_{ij}(t)$  of the particles according to Eqs. (4) and (5);
12:    end for
13:  end for
14:  for  $j = 1 \rightarrow LS$  do
15:    Randomly select two exemplars  $X_{1,k_1}, X_{1,k_2}$  from the first level;
16:    Update the  $X_{2,j}(t)$  and  $V_{2,j}(t)$  of the particles according to Eqs. (4) and (5);
17:  end for
18:  Evaluate the fitness value of every particle and update  $FE_{count}$ ;
19:  Update  $F_g$  and calculate reward  $r_{s_t,a_{t+1}}$  according to Eq. (7);
20:  Update the Q-table with Algorithm 1;
21:  Update  $prob_l$  according to Eq. (9);
22:  Update the state  $s_{t+1} = a_{t+1}$ ;
23: end while

```

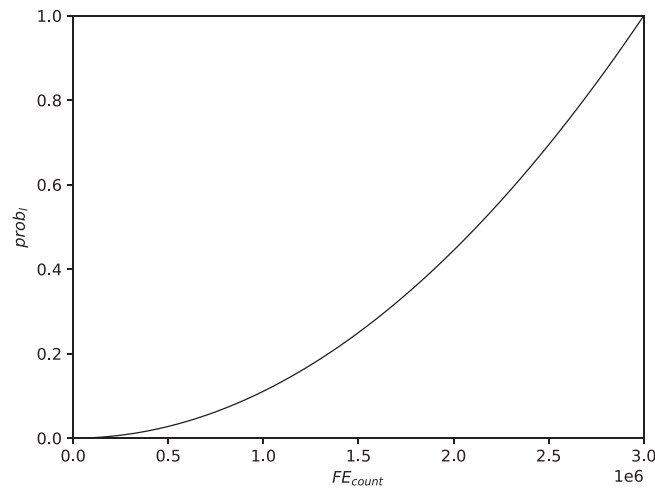


Fig. 2. $prob_i$ changes with FE_{count} .

In Algorithm 3, RLLPSO adopts Algorithm 1 to maintain a Q-table and use it to determine the most effective actions for the current state. The algorithm can improve its search efficiency by self-adaptively adjusting its population structure to the highest potential through the environment rewards. In addition, in the later optimization period, the convergence requirement of the algorithm rises. The level competition mechanism in Algorithm 2 increases the probability of choosing better particles as the exemplars and speeds up the convergence of RLLPSO.

4. Experimental results and comparisons

To verify the effectiveness of RLLPSO, a series of experiments are conducted on 35 test functions from two widely used test suites of LSOPs, i.e., IEEE CEC2010 LSGO test suite with 20 large-scale optimization benchmark functions [27] and IEEE CEC2013 LSGO test suite with 15 large-scale optimization benchmark functions [15].

4.1. Experimental setup

Details of CEC2010 and CEC2013 functions can be found in [15] [27]. In CEC2010, there are three separable functions ($f1$ to $f3$). Additionally, there are 15 partially separable functions ($f4$ to $f18$). And the last 2 functions ($f19$ to $f20$) are non-separable functions. In the CEC2013 test suite, the first three functions ($f21$ to $f23$) are fully separable functions, and the next eight functions ($f24$ to $f31$) are partially separable. The next 3 functions are overlapping functions ($f32$ to $f34$). The last one ($f35$) is the only non-separable function. All of the functions are with 1000 dimensions.

In RLLPSO, the size of the population NP is set at 500, and the parameter ϕ in Eq. (4) is set at 0.4. In the reinforcement learning strategy, the parameters α , γ and ϵ in Eq. (6) and Eq. (8) are set at 0.4, 0.8, 0.9 similar to the regular Q-learning settings. The level number candidate pool is set at $S = \{4, 6, 8, 10, 20, 50\}$.

In addition, the maximum number of fitness evaluations (Max_{FEs}) is set at 3,000,000. All algorithms run 30 times independently for statistics. The mean results and standard deviation (Std) values are calculated and presented. Moreover, Wilcoxon rank-sum test is performed at a significance level of $\alpha = 0.05$ to evaluate the performances of algorithms. All experiments are conducted on a PC with Intel(R) Core(TM) i7-4790 3.60-GHz CPU, 8-GB memory, and Windows 8.1 64-bit system.

4.2. Results comparison and analysis

To comprehensively validate the effectiveness of RLLPSO, we choose five state-of-the-art large-scale optimization algorithms for comparisons. Four of these algorithms are PSO-based algorithms, including DLLSO [41], CCPSO2 [17], SL-PSO [2], and CSO [1]. In the large-scale optimization algorithms, DECC-DG [23] is a famous differential evolution algorithm with a differential grouping strategy. The parameters of the compared algorithms are set as recommended in the corresponding papers and shown in Table 1.

Tables 2 and 3 show the comparison results on convergence accuracy. The best results are highlighted with bold font, and the second-best results are shown with an underline. In the last row, the $w/t/l$ represents the overall performances of RLLPSO against other algorithms.

As seen in Tables 2 and 3, RLLPSO outperforms the other algorithms in most cases. In detail, RLLPSO outperforms dimensionality reduction based algorithms, i.e., DECC-DG and CCPSO2, on 30 and 28 functions out of 35 functions, respectively. From separable functions ($f1$ to $f3$ and $f21$ to $f23$) and some simple partially separable functions ($f4$ to $f8$ and $f24$ to

Table 1
Parameters settings.

Algorithms	Parameters Settings
RLLPSO	$\phi = 0.4, NP = 500$
DLLSO	$\phi = 0.4, NP = 500$
CCPSO2	$p = 0.5, NP = 30$
SL-PSO	$\epsilon = 0.1, \alpha = 0.5, \beta = 0.01, NP = 200$
CSO	$\phi = 0.1, NP = 500$
DECC-DG	$\epsilon = 0.001, NP = 50$

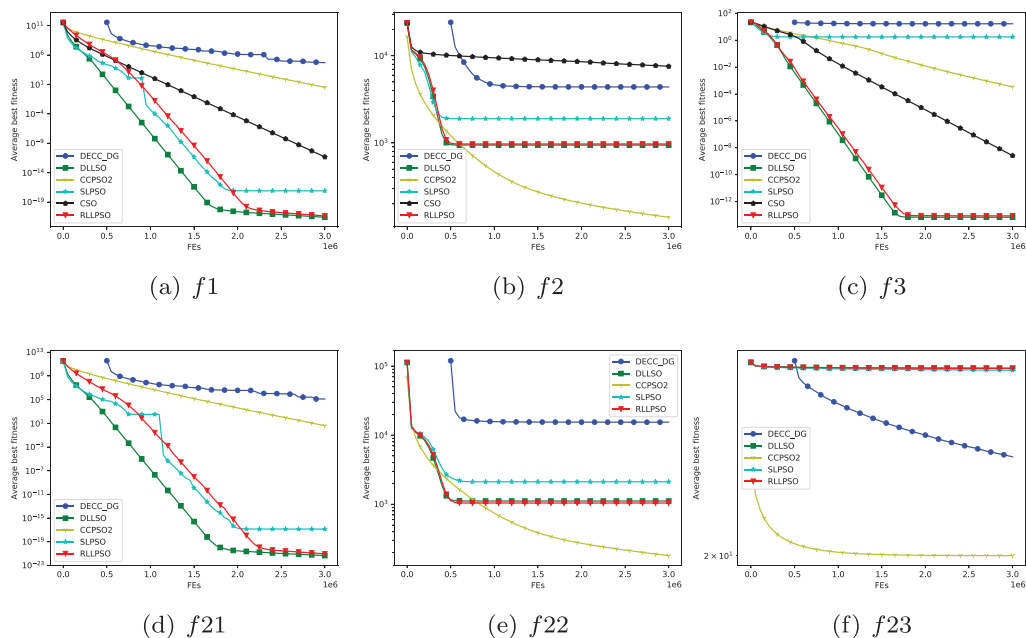
Table 2
Experimental results on CEC2010.

		RLLPSO	DECC-DG	DLLSO	CCPSO2	SL-PSO	CSO
<i>f</i> 1	Mean	<u>4.48E−22</u>	5.04E+04	2.68E−22	3.25E+00	8.26E−18	4.70E−12
	Std	<u>1.70E−22</u>	2.26E+05	5.30E−23	4.42E+00	1.51E−18	6.67E−13
<i>f</i> 2	Mean	9.72E+02	4.38E+03	<u>9.45E+02</u>	1.41E+02	1.90E+03	7.55E+03
	Std	<u>5.71E+01</u>	1.55E+02	3.80E+01	1.27E+02	1.37E+02	2.56E+02
<i>f</i> 3	Mean	<u>8.67E−14</u>	1.68E+01	6.87E−14	3.37E−04	1.77E+00	2.60E−09
	Std	<u>1.08E−14</u>	2.02E−01	4.14E−15	6.59E−04	2.86E−01	1.68E−10
<i>f</i> 4	Mean	<u>6.50E+11</u>	3.60E+12	8.09E+11	4.28E+12	5.11E+11	1.31E+12
	Std	5.02E+10	9.78E+11	2.05E+11	2.78E+12	<u>1.76E+11</u>	1.89E+11
<i>f</i> 5	Mean	<u>1.26E+07</u>	9.07E+07	1.29E+07	3.90E+08	2.94E+07	4.22E+06
	Std	2.64E+06	1.83E+07	<u>2.47E+06</u>	6.41E+07	7.03E+06	1.66E+06
<i>f</i> 6	Mean	3.35E−01	3.85E+04	<u>2.95E−01</u>	1.74E+07	1.98E+01	8.03E−07
	Std	<u>5.17E−01</u>	2.07E+05	6.38E−01	4.56E+06	4.03E+00	2.27E−08
<i>f</i> 7	Mean	2.37E−01	2.53E+04	<u>6.33E+00</u>	1.32E+10	6.23E+04	1.15E+04
	Std	1.35E−01	1.30E+04	<u>8.97E+00</u>	1.30E+10	3.96E+04	3.64E+03
<i>f</i> 8	Mean	<u>1.43E+07</u>	4.63E+07	2.33E+07	7.25E+07	1.03E+07	3.86E+07
	Std	1.13E+07	2.64E+07	<u>1.67E+05</u>	4.05E+07	6.45E+06	6.29E+04
<i>f</i> 9	Mean	<u>3.00E+07</u>	5.43E+07	4.03E+07	1.12E+08	2.87E+07	6.25E+07
	Std	1.13E+06	3.52E+06	3.96E+06	2.16E+07	<u>2.90E+06</u>	5.28E+06
<i>f</i> 10	Mean	<u>9.00E+02</u>	4.23E+03	8.82E+02	4.57E+03	2.51E+03	9.60E+03
	Std	<u>5.79E+01</u>	1.33E+02	4.27E+01	3.32E+02	1.58E+02	8.34E+01
<i>f</i> 11	Mean	<u>2.41E+00</u>	1.29E+01	4.96E+00	1.99E+02	2.26E+01	4.23E−08
	Std	3.11E+00	<u>5.78E−01</u>	4.87E+00	4.15E+00	2.00E+00	5.81E−09
<i>f</i> 12	Mean	1.12E+04	1.58E+04	<u>1.45E+04</u>	1.70E+05	1.99E+04	5.39E+05
	Std	6.09E+02	2.23E+03	<u>1.00E+03</u>	1.80E+04	1.21E+04	5.98E+04
<i>f</i> 13	Mean	9.62E+02	8.60E+03	<u>9.65E+02</u>	1.49E+03	1.12E+03	1.08E+03
	Std	1.44E+02	3.87E+03	<u>3.11E+02</u>	6.04E+02	6.29E+02	4.89E+02
<i>f</i> 14	Mean	<u>1.11E+08</u>	3.07E+08	1.33E+08	4.09E+08	8.34E+07	2.87E+08
	Std	2.32E+06	1.39E+07	6.86E+06	1.12E+08	<u>6.80E+06</u>	1.55E+07
<i>f</i> 15	Mean	8.41E+02	6.07E+03	<u>1.44E+03</u>	9.05E+03	1.12E+04	1.00E+04
	Std	4.71E+01	1.50E+02	2.27E+03	4.23E+02	<u>1.06E+02</u>	6.72E+01
<i>f</i> 16	Mean	1.88E+00	<u>1.49E−01</u>	3.20E+00	3.94E+02	2.14E+01	5.88E−08
	Std	3.02E+00	<u>3.82E−01</u>	1.83E+00	2.18E+00	7.01E+00	6.89E−09
<i>f</i> 17	Mean	5.31E+04	1.26E+05	<u>8.15E+04</u>	4.04E+05	8.30E+04	2.26E+06
	Std	1.63E+03	4.44E+03	<u>4.34E+03</u>	1.11E+05	1.33E+04	1.65E+05
<i>f</i> 18	Mean	2.10E+03	3.21E+09	<u>2.58E+03</u>	1.21E+04	3.02E+03	3.05E+03
	Std	3.90E+02	8.81E+08	<u>7.34E+02</u>	7.34E+03	8.75E+02	2.79E+03
<i>f</i> 19	Mean	8.71E+05	2.17E+06	<u>1.79E+06</u>	2.13E+06	5.03E+06	9.85E+06
	Std	2.31E+04	1.22E+05	<u>1.07E+05</u>	3.52E+06	7.37E+05	5.18E+05
<i>f</i> 20	Mean	1.84E+03	9.18E+10	1.70E+03	<u>1.54E+03</u>	1.75E+03	1.01E+03
	Std	1.68E+02	8.99E+09	1.59E+02	1.88E+02	<u>1.07E+02</u>	4.14E+01
w/t/l		-	19/1/0	11/4/5	18/0/2	14/2/4	13/3/4

*f*27), we see that RLLPSO cannot always work the best of all comparable algorithms, and for these functions, the effectiveness of level-based population structure in RLLPSO is not very obvious. As the function non-separability increases, the performance of RLLPSO is continuously optimized, and the effectiveness of the decomposition strategy decreases. Such phenomenon can be seen from *f*4 to *f*18 in CEC2010 and *f*24 to *f*31 in CEC2013. When it comes to integrality strategy based

Table 3
Experimental results on CEC2013

		RLPSO	DECC-DG	DLLSO	CCPSO2	SL-PSO	CSO
f21	Mean	<u>9.01E-22</u>	1.29E+05	4.54E-22	3.99E+00	1.37E-17	7.32E-12
	Std	<u>3.50E-22</u>	5.44E+05	2.13E-22	5.62E+00	7.28E-18	1.17E-12
f22	Mean	<u>1.04E+03</u>	1.54E+04	1.11E+03	1.80E+02	2.11E+03	8.57E+03
	Std	7.40E+01	7.55E+02	<u>7.49E+01</u>	1.27E+02	1.55E+02	1.60E+02
f23	Mean	2.16E+01	<u>2.08E+01</u>	2.16E+01	2.00E+01	2.16E+01	2.16E+01
	Std	6.21E-03	1.02E-02	<u>4.19E-03</u>	5.42E-04	2.80E-02	6.57E-03
f24	Mean	<u>4.44E+09</u>	1.79E+11	6.30E+09	3.10E+10	4.27E+09	1.30E+10
	Std	4.99E+08	5.88E+10	1.40E+09	1.27E+10	<u>1.25E+09</u>	2.60E+09
f25	Mean	7.25E+05	5.34E+06	<u>6.69E+05</u>	1.48E+07	8.26E+05	5.88E+05
	Std	1.01E+05	4.43E+05	1.09E+05	4.55E+06	1.13E+05	<u>1.02E+05</u>
f26	Mean	1.06E+06	1.06E+06	1.06E+06	1.04E+06	1.06E+06	1.06E+06
	Std	9.95E+02	1.41E+03	6.45E+02	6.87E+03	<u>9.89E+02</u>	1.46E+03
f27	Mean	9.84E+05	1.35E+09	1.61E+06	8.28E+08	<u>1.57E+06</u>	5.10E+06
	Std	2.47E+05	3.66E+08	8.14E+05	1.25E+09	<u>6.79E+05</u>	1.77E+06
f28	Mean	8.56E+13	7.79E+15	1.19E+14	2.34E+15	<u>9.42E+13</u>	2.44E+14
	Std	1.07E+13	2.08E+15	4.18E+13	2.00E+15	<u>4.23E+13</u>	6.11E+13
f29	Mean	3.85E+07	1.27E+09	1.31E+08	3.74E+09	8.72E+07	<u>7.10E+07</u>
	Std	8.14E+06	1.75E+08	3.84E+07	1.02E+09	<u>2.82E+07</u>	7.15E+07
f30	Mean	9.40E+07	9.38E+07	9.41E+07	9.27E+07	<u>9.32E+07</u>	9.40E+07
	Std	<u>2.15E+05</u>	3.45E+05	1.76E+05	5.91E+05	1.42E+06	2.45E+05
f31	Mean	<u>9.25E+11</u>	9.21E+11	9.30E+11	9.40E+11	9.34E+11	9.29E+11
	Std	<u>8.70E+09</u>	3.55E+09	9.10E+09	2.83E+10	1.70E+10	9.06E+09
f32	Mean	1.87E+03	8.76E+10	1.80E+03	<u>1.49E+03</u>	1.79E+03	1.07E+03
	Std	1.72E+02	1.04E+10	1.94E+02	1.58E+02	<u>1.43E+02</u>	5.31E+01
f33	Mean	2.17E+08	2.38E+10	<u>3.21E+08</u>	7.03E+09	4.07E+08	7.97E+08
	Std	5.46E+07	5.72E+09	<u>1.51E+08</u>	2.71E+09	2.41E+08	2.78E+08
f34	Mean	5.06E+07	1.16E+11	<u>1.12E+08</u>	2.31E+11	6.21E+08	5.10E+09
	Std	2.16E+07	3.49E+10	<u>9.18E+07</u>	1.34E+11	7.52E+08	6.13E+09
f35	Mean	1.63E+06	1.37E+07	<u>4.37E+06</u>	1.79E+07	5.89E+07	7.83E+07
	Std	5.88E+04	2.11E+06	<u>2.98E+05</u>	2.07E+07	7.68E+06	7.03E+06
w/t/l		-	11/1/3	10/3/2	10/0/5	9/3/3	10/1/4

**Fig. 3.** Convergence process on separable functions.

algorithms, i.e., DLLSO, SL-PSO, and CSO, RLLPSO outperforms them on 21, 23, 23 functions out of 35 functions, respectively, which can be attributed to the influence of the reinforcement learning strategy for level number control and the level competition mechanism. When a reward is obtained, the algorithm will not change its population structure immediately but

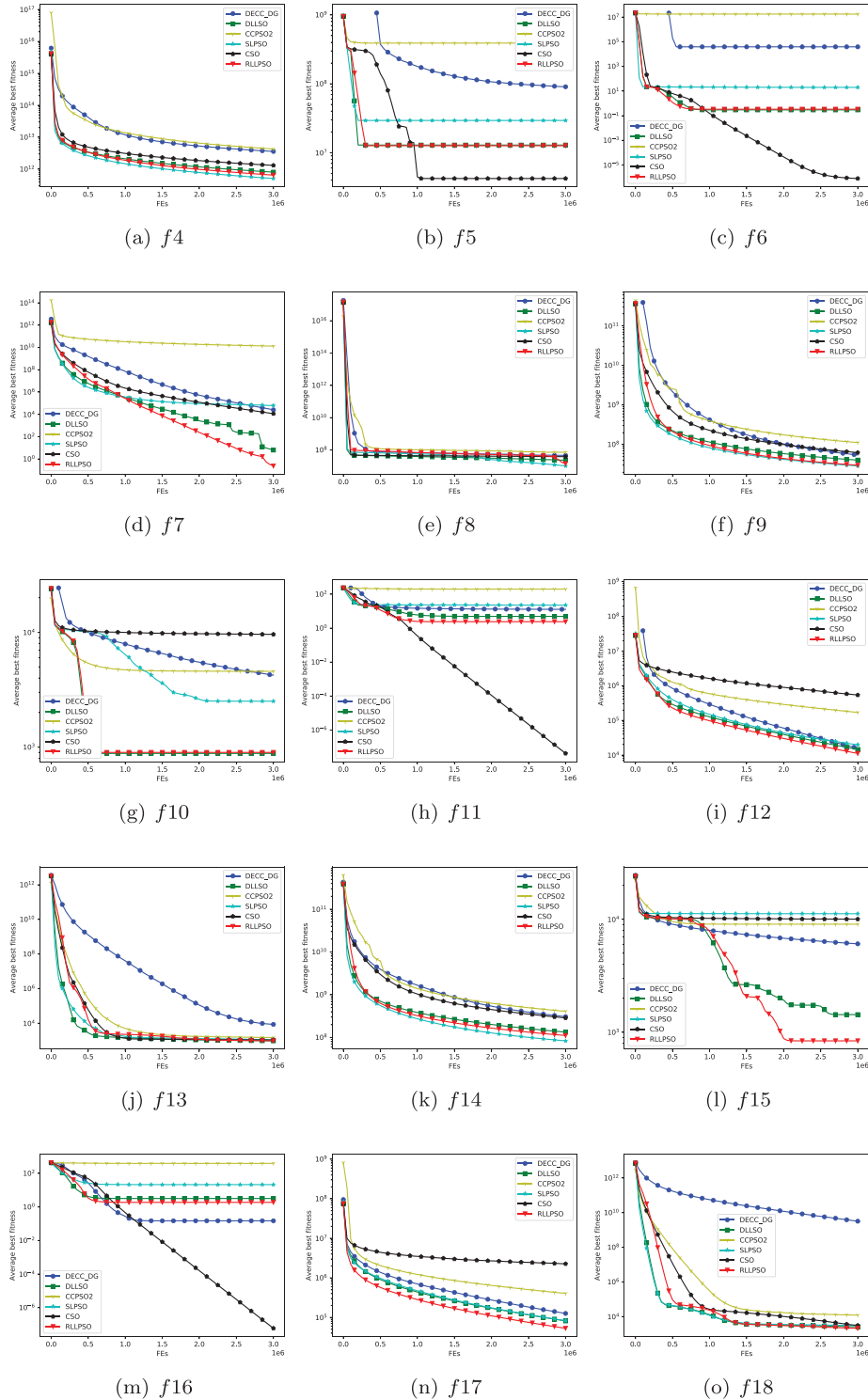


Fig. 4. Convergence process on partially separable functions in CEC2010.

combine it with the historical information to make a decision. The judgments based on both historical experience and instant reward guide the algorithm to take the most efficient actions. Compared with DLLSO, which adjusts its level structure of the population only based on instant reward, RLLPSO shows better results. It means that the reinforcement learning mechanism for level number control is helpful to improve search efficiency. And the level competition mechanism helps RLLPSO obtain better convergence accuracy by leading the particles to learn from more elite exemplars in the later optimization stage. In general, we find that RLLPSO achieves the best results on 13 out of 35 functions and achieves 13 second-best results, which means RLLPSO has the best convergence accuracy on both CEC2010 and CEC2013 test suites.

To further verify the convergence speed of RLLPSO, we also plot figures that represent the convergence process of the algorithms. Fig. 3 shows the convergence process on fully separable functions. Fig. 4 and Fig. 5 represent the convergence process on partially separable functions. Fig. 6 gives the convergence process of overlapping functions. And Fig. 7 shows the convergence performances on non-separable functions. It is worth noticing that due to a different initialize strategy of CCPSO2 and the computation resources consumption of DECC-DG on the group dividing before the optimization begins, these two algorithms may start their convergence curve differently from others.

These figures show that, compared with others, RLLPSO has similar results on most separable functions, but it performs better on partially separable and non-separable functions. It might be because the reinforcement learning mechanism ensures the search efficiency of RLLPSO when the functions become more complex and more difficult to be optimized. More precisely, Fig. 3 indicates that RLLPSO has the second-best or third-best convergence performances with tiny inferiority on most of the fully separable functions. From Fig. 4 and Fig. 5, we see that, the more non-separable the function is, the better convergence performance RLLPSO has. From f_4 to f_{18} and f_{24} to f_{31} , which increases their non-separability every five and four functions, we find that RLLPSO mostly obtains its best convergence performances on functions with higher non-

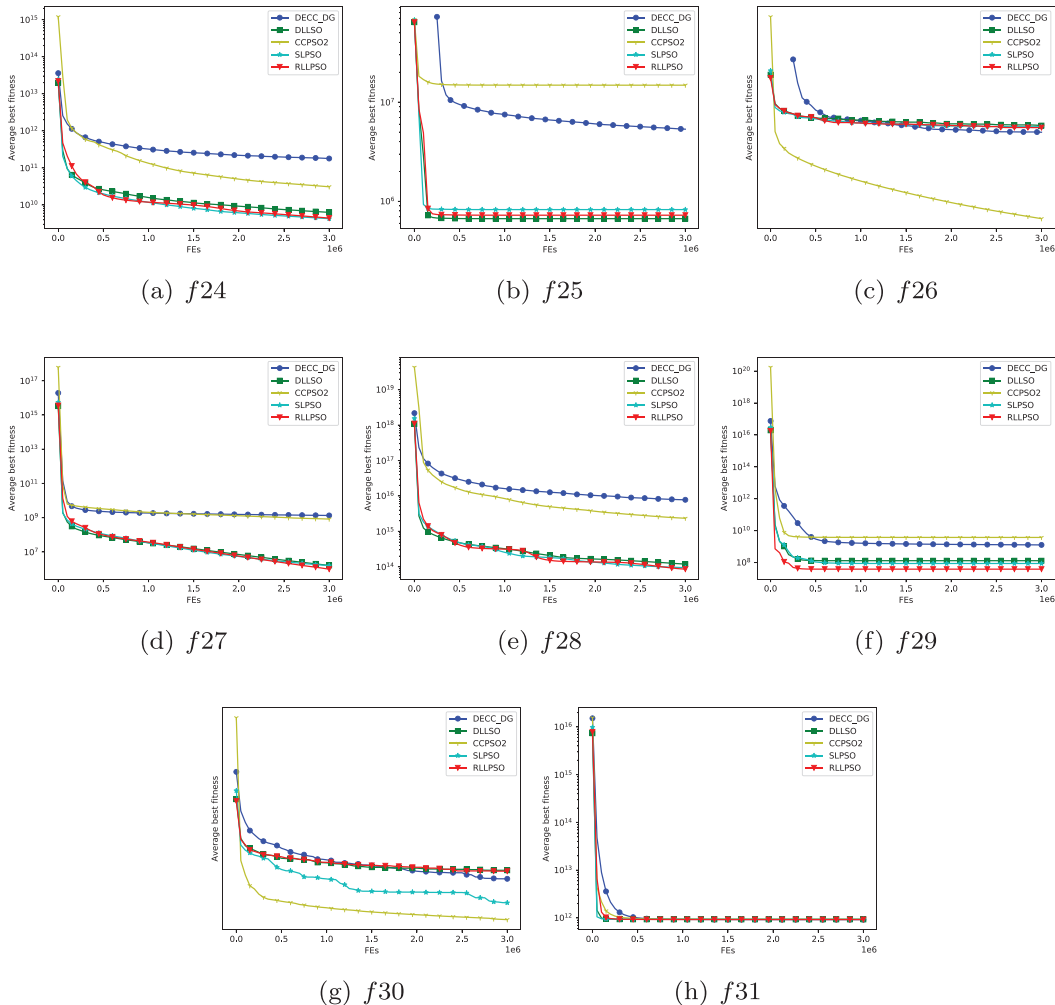


Fig. 5. Convergence process on partially separable functions in CEC2013.

separability. From Fig. 6 and Fig. 7, which represent functions that cannot be decomposed, we find that RLLPSO maintains the best or second-best convergence performance in most cases. For dimensionality reduction based algorithms, their performance deteriorates when the non-separability of the functions increases as the decomposition strategies cannot work correctly. As for integrality strategy based algorithms, increasingly complex functions challenge their search efficiency. The search efficiency and convergence speed of RLLPSO are guaranteed by the reinforcement learning mechanism and the level competition mechanism. RLLPSO can compete with dimensionality reduction based algorithms on separable functions while showing superior performances on more complex functions that are partially separable or non-separable.

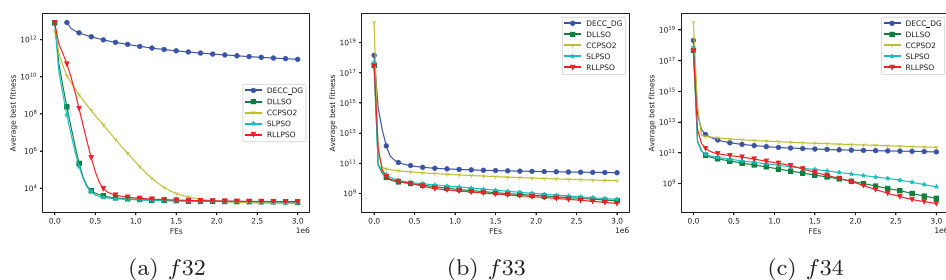


Fig. 6. Convergence process on overlapping functions.

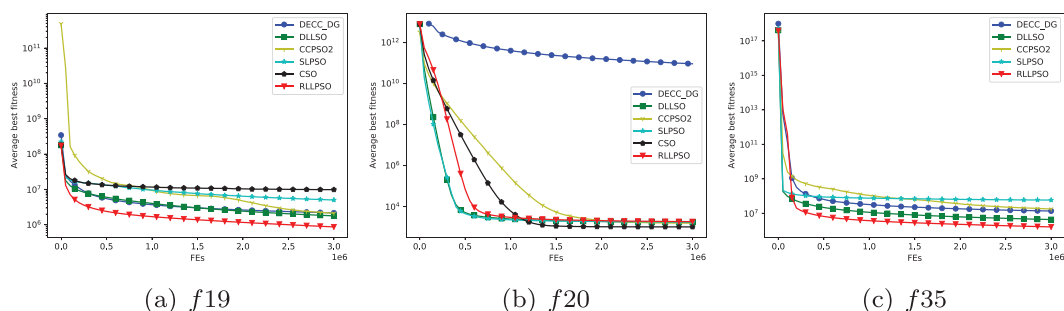


Fig. 7. Convergence process on non-separable functions.

Table 4
Experimental results of RLLPSO and RLLPSO-WO on CEC2010

		RLLPSO	RLLPSO-WO			RLLPSO	RLLPSO-WO
f1	Mean	4.48E-22	8.31E-22	f11	Mean	2.41E+00	1.86E+00
	Std	1.70E-22	2.09E-22		Std	3.11E+00	1.70E+00
f2	Mean	9.72E+02	9.63E+02	f12	Mean	1.12E+04	1.14E+04
	Std	5.71E+01	6.92E+01		Std	6.09E+02	4.21E+02
f3	Mean	8.67E-14	7.58E-14	f13	Mean	9.62E+02	9.78E+02
	Std	1.08E-14	5.53E-15		Std	1.44E+02	1.32E+02
f4	Mean	6.50E+11	6.72E+11	f14	Mean	1.11E+08	1.09E+08
	Std	5.02E+10	4.46E+10		Std	2.32E+06	2.76E+06
f5	Mean	1.26E+07	1.33E+07	f15	Mean	8.41E+02	1.14E+03
	Std	2.64E+06	2.48E+06		Std	4.71E+01	1.62E+03
f6	Mean	3.35E-01	3.43E-01	f16	Mean	1.88E+00	1.47E+00
	Std	5.17E-01	5.31E-01		Std	3.02E+00	2.17E+00
f7	Mean	2.37E-01	6.07E-01	f17	Mean	5.31E+04	5.32E+04
	Std	1.35E-01	3.35E-01		Std	1.63E+03	1.46E+03
f8	Mean	1.43E+07	3.47E+07	f18	Mean	2.10E+03	2.01E+03
	Std	1.13E+07	2.65E+07		Std	3.90E+02	2.74E+02
f9	Mean	3.00E+07	3.05E+07	f19	Mean	8.71E+05	9.73E+05
	Std	1.13E+06	1.35E+06		Std	2.31E+04	1.80E+04
f10	Mean	9.00E+02	8.90E+02	f20	Mean	1.84E+03	1.86E+03
	Std	5.79E+01	6.82E+01		Std	1.68E+02	2.16E+02
w/t/l				6/12/2			

From figures of functions such as $f1, f7, f9, f12, f15, f17, f18, f21, f27, f28$, and $f34$, we see that even the performances of RLLPSO are rather mediocre in the early optimization stage, RLLPSO can gradually outperform others with its consistent improving converge ability brought by the level competition mechanism. At the early optimization stage, RLLPSO mainly focuses on fully exploring the massive search space by random exemplar selection, which results in poor convergence performance. However, the probability of the level competition mechanism being triggered keeps increasing during the optimization. And the higher quality learning exemplars increase the convergence speed, which makes RLLPSO gradually outperforms other compared algorithms in the later optimization stage. Nevertheless, it should also be noticed that when the optimization process converges at the early stage, such as $f2, f5, f8$, and $f16$, the effectiveness of the level competition mechanism is affected as it is rarely triggered. Overall, we conclude that RLLPSO shows competitive convergence speed performance and strong convergence accuracy performance, which outperform the other five competitors.

4.3. Effects of level competition mechanism

To further verify the effectiveness of the proposed level competition mechanism, we compare RLLPSO with an algorithm not using this mechanism (named as RLLPSO-WO). As the comparisons between RLLPSO and DLLSO have shown the superiority of the reinforcement learning strategy for level number control, the effectiveness of the reinforcement learning strategy will not be further discussed.

RLLPSO-WO is tested on CEC2010, and the results are compared with the normal RLLPSO and shown in Table 4. The results which are better than the other with a significance level of $\alpha = 0.05$ are presented in bold font.

From Table 4, we can confirm that although the convergence accuracy performances of 12 functions are not severely influenced and two of them retrogress, the level competition mechanism still improves the performances of 6 functions from all three groups. From the results, we can see that the level competition mechanism can improve the convergence accuracy of RLLPSO with few extra computation costs.

Additionally, the level competition mechanism can also help improve convergence speed of the algorithm. As the level competition mechanism is rarely invoked during the early stage and the population of the algorithm tends to convergence

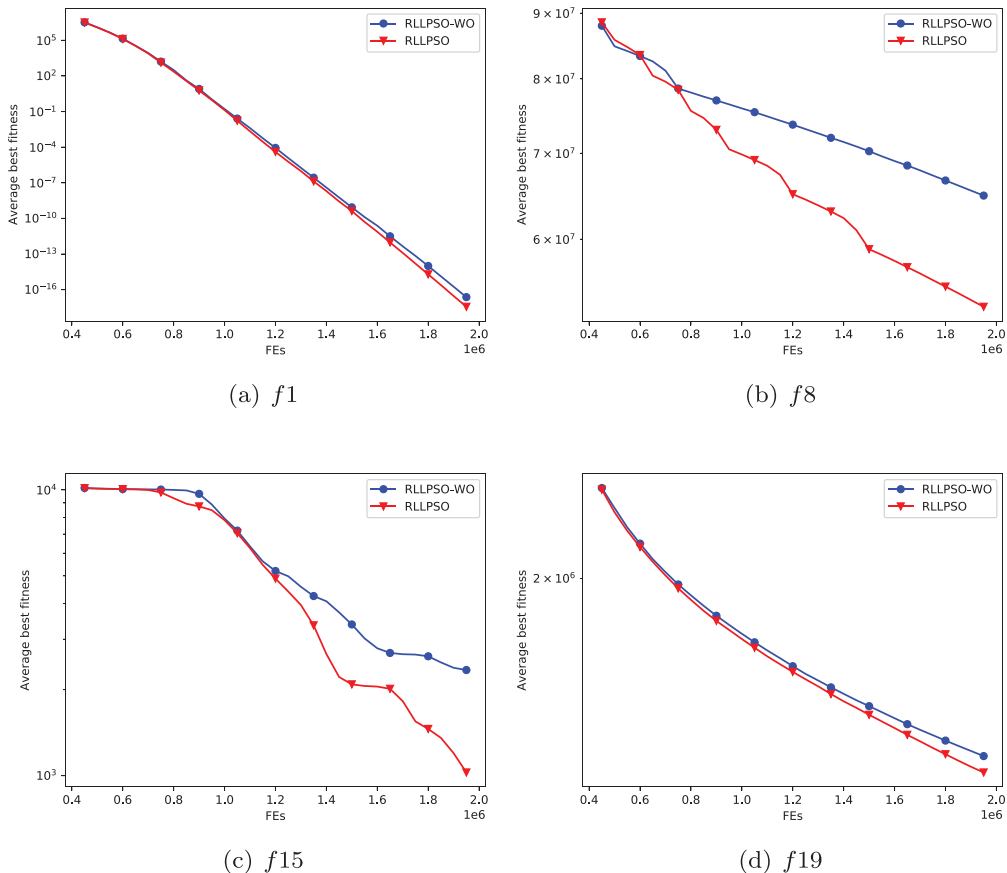


Fig. 8. Convergence process of RLLPSO and RLLPSO-WO on selected functions.

in the final stage, we mainly focus on the middle stage convergence process. The convergence process of RLLPSO and RLLPSO-WO in $f1, f8, f15$, and $f19$ from all three function categories of CEC2010 are plotted and shown in Fig. 8.

From Fig. 8, we see that although with a similar result at the beginning, RLLPSO gradually outperforms RLLPSO-WO in the later stage of the convergence process. From Fig. 8 and Table 4, it is evident that RLLPSO can converge not only more accurately but also faster, which can be attributed to the level competition mechanism that can speed up the convergence process.

5. Conclusion and future work

In this paper, a new large-scale optimization algorithm named RLLPSO is proposed. To improve the exploration ability of traditional PSO and avoid its premature inclination, we adopt the level-based population structure from LLSO. Furthermore, we also propose a reinforcement learning strategy for level number control to self-adaptively adjust the level number of the population. Based on the environment reward, RLLPSO can select the proper level structure to maximize the search potential of the population and increase search efficiency. To handle the massive search space, RLLPSO introduces a level competition mechanism to speed up the convergence process by giving the elite particles a higher probability of being selected as the learning exemplars. The experimental results show that RLLPSO outperforms other state-of-the-art large-scale optimization algorithms on convergence accuracy and speed.

In the future, we will further explore the potential of the reinforcement learning mechanism in EAs, such as making it participate in the optimization process. Additionally, LSOPs with constraints have always been a challenging research field due to their highly irregular feasible solution space. Reinforcement learning, which has the ability to interact with the environment, may be a promising approach. Furthermore, due to the resemblance between LSOPs and real-world problems, the real-world application of RLLPSO is also worth investigating.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work is supported by National Nature Science Foundation of China [Grant Nos. 62173258, 61773296].

References

- [1] R. Cheng, Y. Jin, A competitive swarm optimizer for large scale optimization, *IEEE Trans. Cybern.* 45 (2015) 191–204.
- [2] R. Cheng, Y. Jin, A social learning particle swarm optimization algorithm for scalable optimization, *Inf. Sci.* 291 (2015) 43–60.
- [3] H. Deng, L. Peng, H. Zhang, B. Yang, Z. Chen, Ranking-based biased learning swarm optimizer for large-scale optimization, *Inf. Sci.* 493 (2019) 120–137.
- [4] M.M. Drugan, Reinforcement learning versus evolutionary computation: A survey on hybrid algorithms, *Swarm Evolutionary Comput.* 44 (2019) 228–246.
- [5] R. Eberhart, J. Kennedy, A new optimizer using particle swarm theory, in: *Mhs95 Sixth International Symposium on Micro Machine & Human Science*, 1995, pp. 1942–1948..
- [6] V.D.B. Frans, A.P. Engelbrecht, A cooperative approach to particle swarm optimization, *IEEE Trans. Evol. Comput.* 8 (2004) 225–239.
- [7] Huang Han, Lv Liang, Ye Shujin, Hao Zhifeng, . article swarm optimization with convergence speed controller for large-scale numerical optimization. *Soft Comput. – A Fusion Foundations, Methodol. Appl.*, 2018, 4421–4437..
- [8] Y.H. Jia, W.N. Chen, T. Gu, H. Zhang, H.Q. Yuan, S. Kwong, J. Zhang, Distributed cooperative co-evolution with adaptive computing resource allocation for large scale optimization, *IEEE Trans. Evol. Comput.* 23 (2019) 188–202.
- [9] J.R. Jian, Z.G. Chen, Z.H. Zhan, J. Zhang, Region encoding helps evolutionary computation evolve faster: A new solution encoding scheme in particle swarm for large-scale optimization, *IEEE Trans. Evol. Comput.* 25 (2021) 779–793.
- [10] C.F. Juang, T.B. Bui, Reinforcement neural fuzzy surrogate-assisted multiobjective evolutionary fuzzy systems with robot learning control application, *IEEE Trans. Fuzzy Syst.* 28 (2020) 434–446.
- [11] G. Karafotias, A.E. Eiben, M. Hoogendoorn, Generic parameter control with reinforcement learning, *Assoc. Comput. Mach.* (2014) 1319–1326.
- [12] K. Li, Y. Chen, W. Li, J. He, Y. Xue, Improved gene expression programming to solve the inverse problem for ordinary differential equations, *Swarm Evolut. Comput.* 38 (2018) 231–239.
- [13] K. Li, Z. Liang, S. Yang, Z. Chen, Z. Lin, Performance analyses of differential evolution algorithm based on dynamic fitness landscape, *Int. J. Cognit. Inform. Natural Intell.* 13 (2019) 36–61.
- [14] K. Li, H. Wang, S. Li, A mobile node localization algorithm based on an overlapping self-adjustment mechanism, *Inf. Sci.* 481 (2019) 635–649.
- [15] X. Li, K. Tang, M.N. Omidvar, Z. Yang, K. Qin, Benchmark functions for the CEC'2013 special session and competition on large-scale global optimization, 2013..
- [16] X. Li, X. Yao, Tackling high dimensional nonseparable optimization problems by cooperatively coevolving particle swarms, in: *2009 IEEE Congress on Evolutionary Computation*, 2009, pp. 1546–1553..
- [17] X. Li, X. Yao, Cooperatively coevolving particle swarms for large scale optimization, *IEEE Trans. Evol. Comput.* 16 (2012) 210–224.
- [18] H. Liu, Y. Wang, N. Fan, A hybrid deep grouping algorithm for large scale global optimization, *IEEE Trans. Evol. Comput.* 24 (2020) 1112–1124.
- [19] Y. Liu, H. Lu, S. Cheng, Y. Shi, An adaptive online parameter control algorithm for particle swarm optimization based on reinforcement learning, in: *2019 IEEE Congress on Evolutionary Computation (CEC)*, 2019, pp. 815–822..
- [20] Y. Mei, M.N. Omidvar, X. Li, X. Yao, A competitive divide-and-conquer algorithm for unconstrained large-scale black-box optimization, *ACM Trans. Math. Softw.* (2016) 42.
- [21] P. Mohapatra, K.N. Das, S. Roy, An improvised competitive swarm optimizer for large-scale optimization, in: *Soft Computing for Problem Solving*, Springer, Singapore, 2019, pp. 591–601.

- [22] W. Ning, B. Guo, X. Guo, C. Li, Y. Yan, Reinforcement learning aided parameter control in multi-objective evolutionary algorithm based on decomposition, *Progress in Artificial Intelligence* 7 (2018) 385–398.
- [23] M.N. Omidvar, X. Li, Y. Mei, X. Yao, Cooperative co-evolution with differential grouping for large scale optimization, *IEEE Trans. Evol. Comput.* 18 (2014) 378–393.
- [24] M.A. Potter, K.A. De Jong, A cooperative coevolutionary approach to function optimization, in: *Parallel Problem Solving from Nature – PPSN III*, Springer, Berlin Heidelberg, 1994, pp. 249–257.
- [25] T. Ray, X. Yao, A cooperative coevolutionary algorithm with correlation based adaptive variable partitioning, in: *2009 IEEE Congress on Evolutionary Computation*, 2009, pp. 983–989..
- [26] H. Samma, C.P. Lim, J. Mohamad Saleh, A new reinforcement learning-based memetic particle swarm optimizer, *Appl. Soft Comput.* 43 (2016) 276–297.
- [27] K. Tang, X. Li, P.N. Suganthan, Z. Yang, T. Weise, Benchmark functions for the CEC'2010 special session and competition on large-scale global optimization, *Nature Inspired Comput. Appl. Labor.* (2010).
- [28] Y. Tian, X. Zheng, X. Zhang, Y. Jin, Efficient large-scale multiobjective optimization based on a competitive swarm optimizer, *IEEE Trans. Cybern.* 50 (2020) 3696–3708.
- [29] F. Wang, Y. Li, F. Liao, H. Yan, An ensemble learning based prediction strategy for dynamic multi-objective optimization, *Appl. Soft Comput.* 96 (2020) 106592.
- [30] F. Wang, Y. Li, A. Zhou, K. Tang, An estimation of distribution algorithm for mixed-variable newsvendor problems, *IEEE Trans. Evol. Comput.* 24 (2020) 479–493.
- [31] F. Wang, F. Liao, Y. Li, H. Wang, A new prediction strategy for dynamic multi-objective optimization using gaussian mixture model, *Inf. Sci.* 580 (2021) 331–351.
- [32] F. Wang, H. Zhang, K. Li, Z. Lin, J. Yang, X.L. Shen, A hybrid particle swarm optimization algorithm using adaptive learning strategy, *Inf. Sci.* 436–437 (2018) 162–177.
- [33] F. Wang, H. Zhang, A. Zhou, A particle swarm optimization algorithm for mixed-variable optimization problems, *Swarm Evolut. Comput.* 60 (2021) 100808.
- [34] H. Wang, M. Liang, C. Sun, G. Zhang, L. Xie, Multiple-strategy learning particle swarm optimization for large-scale optimization problems, *Complex Intell. Syst.* (2020) 1–16.
- [35] L. Wang, Z. Pan, J. Wang, A review of reinforcement learning based intelligent optimization for manufacturing scheduling, *Complex Syst. Model. Simul.* 1 (2021) 257–270.
- [36] Z.J. Wang, Z.H. Zhan, S. Kwong, H. Jin, J. Zhang, Adaptive granularity learning distributed particle swarm optimization for large-scale optimization, *IEEE Trans. Cybern.* 51 (2021) 1175–1188.
- [37] Z.J. Wang, Z.H. Zhan, W.J. Yu, Y. Lin, J. Zhang, T.L. Gu, J. Zhang, Dynamic group learning distributed particle swarm optimization for large-scale optimization and its application in cloud workflow scheduling, *IEEE Trans. Cybern.* 50 (2020) 2715–2729.
- [38] O. Watchanupaporn, P. Pudtuan, Multi-robot target reaching using modified q-learning and pso, in: *2016 2nd International Conference on Control, Automation and Robotics (ICCAR)*, 2016, pp. 66–69..
- [39] C. Watkins, P. Dayan, Q-learning, *Mach. Learn.* 8 (1992) 279–292.
- [40] Y. Xu, D. Pi, A reinforcement learning-based communication topology in particle swarm optimization, *Neural Comput. Appl.* 32 (2020) 10007–10032.
- [41] Q. Yang, W.N. Chen, J.D. Deng, Y. Li, T. Gu, J. Zhang, A level-based learning swarm optimizer for large-scale optimization, *IEEE Trans. Evol. Comput.* 22 (2018) 578–594.
- [42] Q. Yang, W.N. Chen, T. Gu, H. Zhang, J.D. Deng, Y. Li, J. Zhang, Segment-based predominant learning swarm optimizer for large-scale optimization, *IEEE Trans. Cybern.* 47 (2017) 2896–2910.
- [43] X. Zhang, K.J. Du, Z.H. Zhan, S. Kwong, T.L. Gu, J. Zhang, Cooperative coevolutionary bare-bones particle swarm optimization with function independent decomposition for large-scale supply chain network design with uncertainties, *IEEE Trans. Cybern.* 50 (2020) 4454–4468.
- [44] X.Y. Zhang, J. Zhang, Y.J. Gong, Z.H. Zhan, W.N. Chen, Y. Li, Kuhn-munkres parallel genetic algorithm for the set cover problem and its application to large-scale wireless sensor networks, *IEEE Trans. Evol. Comput.* 20 (2016) 695–710.
- [45] Y.F. Zhang, H.D. Chiang, A novel consensus-based particle swarm optimization-assisted trust-tech methodology for large-scale global optimization, *IEEE Trans. Cybern.* 47 (2017) 2717–2729.
- [46] S.Z. Zhao, J.J. Liang, P.N. Suganthan, M.F. Tasgetiren, Dynamic multi-swarm particle swarm optimizer with local search for large scale global optimization, in: *2008 IEEE Congress on Evolutionary Computation (IEEE World Congress on Computational Intelligence)*, 2008, spp. 3845–3852..